

2048 Game

Melih Efe Sönmez
İlhan Altınay

1 Project Overview

This project is an implementation of classic 2048 sliding-tile puzzle by QT framework. Main aim of the game is to merge the tiles (which have the same values on it) on a grid and to obtain 2048 tile.

There are three modes of this game:

- **Normal Mode:** The mission is reaching 2048 tile.
- **Unlimited Mode:** The game does not finish when 2048 tile is obtained.
- **Hard Mode:** In this mode, player should slide in five seconds; otherwise, a random slide is executed.

The game is initiated with Normal Mode, and the program executes inputs (which are slides, restart, undo, and changing mode) by QT's event-driven architecture. Also, the program shows required widgets (score, best score, mode names, win/lose messages, timer) until program is closed.

2 User Interface Design

The structure of UI is based on `MainWindow` class, which is inherited from `QWidget` class. Moreover, the general scheme of the widgets is flexible thanks to `QVBoxLayout`, `QHBoxLayout` and `QGridLayout`.

UI has three parts vertically:

- **Header:** It stores game heading&subheading, score, and best score. These are lined by `QHBoxLayout`.

- **Mode and Feature Buttons:** It stores mode selection buttons by `QButtonGroup`, undo, and restart buttons. Additionally, a timer label is here; however, it is only shown in hard mode.
- **Grid:** Grid of the game is a `QFrame`, which has `QGridLayout`. The tiles are kept as `QLabel` in an one-dimensional `std::vector<QLabel*>` to achieve $O(1)$ time complexity while UI is updating.

There is a hidden overlay, which is also a `QFrame`, for the end of the game. Within this frame, there is a `QLabel` (which shows end of the game message), and there is a `QPushButton` for restarting the game. This overlay is hidden until the end of the game, and it is made visible by `overlay->show()` at the end.

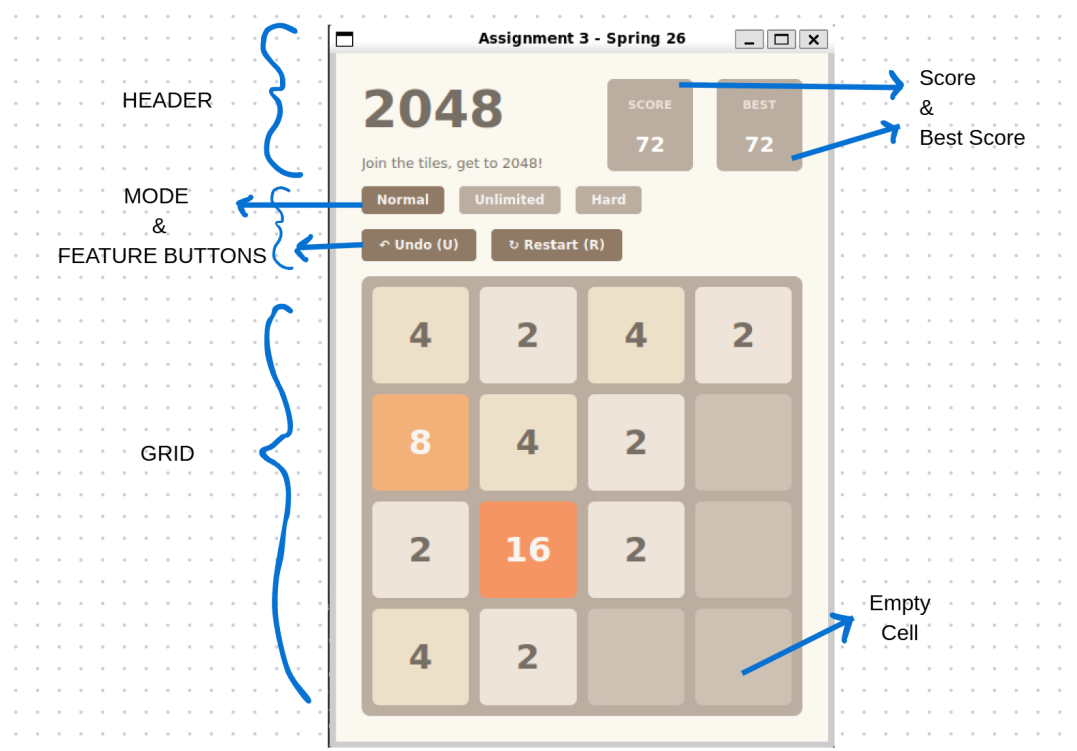


Figure 1: Screenshot of User Interface

3 Game State Representation

Game state is managed by `GameLogic` class, which is independent from user interface.

- The board is stored by `std::vector<std::vector<int>> grid_`, which is a dynamic array. Sizing of board the grid is done by `rows_` and `cols_` parameters (4x4 in default). Empty cells are represented by value 0. `emptyCells()` function is coded to detect empty cells, which is required for random tile placement.
- Tile values are managed by `slideAndMerge` function. This function adjust the line according to sliding direction and gives it to `slideAndMergeLine`, and the line is updated by merging equal tiles in front.

- The target value is checked by `hasWon()` function. This function controls every tile in the grid for equality with the tile value and the target value.
- `score_`: Every merging two tiles' sum is added the score.
- `bestScore_`: It is stored `MainWindow` class. After every valid move, best score and board's score are compared and updated if necessary. It is not affected by undo move, and it keeps the best score of the current session.
- `history_`: For Undo, `std::vector<std::pair<std::vector<std::vector<int>>, int>>` `history_` is used in `GameLogic` class.

When the player make a valid slide, `snapshotOfScore` and `snapshotOfGrid` is added to `history_` by `push_back` function. When "undo" is pressed, `undo()` function is activated. Firstly, `history_` is checked whether empty or not. If it is not empty, last element of the vector is obtained by `back()`, and `grid_` and `score_` are updated. At the end, the last element is removed from the vector by `pop_back`.

4 Event-Driven Architecture and Game Logic

The program starts event loop by `app.exec()` in `main.cpp`, and waits for the player's inputs and timer signals. These signals are detected by `MainWindow` class.

4.1 Keyboard and Button Events

Keyboard inputs are processed by overridden `keyPressEvent(QKeyEvent *event)` in `MainWindow`.

- **Slider ('WASD' & Arrow Keys), Restart ('R'), and Undo ('U') Inputs:** WASD and arrows are used for sliding moves, and direction is sent to `applyMove()` function when one of them is pressed. Moreover, 'U' key runs `onUndo()`, and 'R' key runs `onRestart()`.
- **Invalid Moves:** Nothing changes if move is invalid, and it is accepted as no input, which means history or another things are not affected.
- **Input Disabling:** Keyboard inputs are disabled by making `inputLocked_ true` after win/loss conditions.

4.2 Signals, Slots, and Connections

QT signal-slot mechanism is used to make connections between UI components, timer and game logic. These connections are handled in `MainWindow::buildUi()` function.

Signal	Slot / Function	Purpose
restartButton_ -> clicked()	onRestart()	Reset board, score and start new game
undoButton_ -> clicked()	onUndo()	Update the game to previous state
modeGroup_ -> idClicked(int)	onModeChanged(int)	Update game mode
hardTimer_ -> timeout()	onHardTimeout()	Make a random valid move in 5 seconds
countdownTimer_ -> timeout()	updateHardCountdown()	Update countdown text 10 times in a second in hard mode
overlayButton_ -> clicked()	onOverlayButton()	End or restart game

Table 1: Main Qt signal-slot connections used in the project

In the Table 1, **Signal** column represents the objects which emits the signal. **Slot** column represents to be run functions. **Purpose** column explains the state changes.

In short, when one of the object are triggered, object emits the signal. **MainWindow** captures the signal, and runs the appropriate slot.

4.3 Move and Merge Logic

A single move is processed by `GameLogic::slideAndMerge()` function. Grid is converted to `std::vector<int>` line by line by according to the slide direction, and sent to `slideAndMergeLine()`.

The working mechanism of `slideAndMergeLine()`:

- **Compact:** A compact vector is obtained by removing zeros.
- **Merge:** Iteration is done on the compact vector. The adjacent tiles with same values are pushed to merged vector as merged, the others directly pushed. As a result of merging sum of the two tiles merged are added gained, which represents that slide's gain. To prevent illegal doubles, loop increments in merging condition one more as an extra. Merge ordering is also handled automatically by iteration.
- **Pad:** Merged vector is padded by zeros to resize it to original size.

After the move, `score_` is updated and `spawnRandomTile()` is called to place a 2 or 4 tile to an empty cell.

4.4 Timer-Based Events

Hard Mode is built with a Qt timer. In **MainWindow** we keep a `QTimer` called `hardTimer_` which is set to 5000 ms. When the player switches to Hard Mode or makes move in it, we

start timer. If it reaches the end without the player moving, it emits `timeout()` signal, which is connected to `onHardTimeout()` slot.

Inside `onHardTimeout()` we do not pick a random direction, we ask model for `validMoves()`, which returns only the directions that change the board, and then we choose one of them at random.

The timer is reset after every successful action. Our function `restartHardTimerIfNeeded()` is called at the end of each move, so the five seconds always start counting from the player's last action. When the mode is not Hard, or the game has ended, the timer is stopped.

We also added a visual countdown. A second timer, `countdownTimer_`, repeats every 100 ms and only updates a label that shows the remaining time, for example "Auto move in 4.1 s". It reads `hardTimer->remainingTime()` and does not affect the game; it is only there so the player can see how much time is left. The label is shown only in Hard Mode and hidden in the other modes.

5 Code Structure

The project is split into two parts so that the game rules and the user interface are separate.

- **main.cpp:** The entry point. It creates the `QApplication`, creates one `MainWindow`, shows it, and starts the event loop with `app.exec()`.
- **GameLogic (gamelogic.h, gamelogic.cpp):** The model. This class is C++ and does not use Qt. It stores the grid, the score and the undo history, and the rules: sliding, merging, spawning new tiles, and checking win/lose. Its main functions are `move()`, `undo()`, `restart()`, `slideAndMerge()`, `spawnRandomTile()`, `hasWon()`, `isGameOver()` and `validMoves()`.
- **MainWindow (mainwindow.h, mainwindow.cpp):** The view and controller. It inherits from `QWidget`, builds all the widgets and layouts, and handles input. It keeps a `GameLogic` object as a member, calls its functions when the user does something, and then refreshes the screen with `updateView()`.

We chose this division because it keeps the rules in one place and the interface in another. The logic can be understood without reading any Qt code, and changing the look of the game does not require touching the game rules.

6 Testing and Debugging

We tested the game by playing it and by checking some specific cases:

- **Repeated equal tiles:** A line like `[2,2,2,2]` becomes `[4,4]` and `[2,2,2]` becomes `[4,2]`, not `[8]`. Each tile merges only once per move.

- **Invalid move:** If a slide does not move or merge any tile, nothing happens and no new tile appears.
- **Undo at the start:** If there is no history yet, pressing Undo does nothing.
- **Full board:** When the board is full and no merge is possible, the game shows "Game Over".
- **Winning in each mode:** Reaching the target shows "You Win!" in every mode, in Unlimited Mode the player can keep playing.
- **Mode switching:** Changing the mode keeps the same board and score.

To test the end states faster, we temporarily lowered the target value K (for example to 4) so winning takes only a few moves, and used a smaller grid so the board fills up quickly to reach Game Over. After the tests we set the values back to the defaults.

7 Challenges and Solutions

Merge order and double merges. The hardest part of the logic was making sure each tile merges at most once and that the correct pair merges first. We solved this in `slideAndMergeLine()`: we first remove the empty cells, then walk through the line once from the border side. When two equal tiles are next we merge them and skip the next index (`++i`) so the merged tile cannot merge again in the same move. Because we scan from the border, the pair closest to the border merges first.

Switching modes without restarting. At first our mode buttons started a new game every time. But the rule is that switching modes should keep the current board and score. We fixed this by separating "start a new game" from "switch mode": `startGame()` resets everything, while `onModeChanged()` only changes the active rules. We also used the flag `inputLocked_`, so the game knows when input should be allowed.

8 Conclusion

The strongest part of our design is the clear separation between the game logic (`GameLogic`) and the interface (`MainWindow`), which made the code easier to write and to understand. The most difficult part was the move and merge logic, especially handling merge order and avoiding double merges. The game could be improved by adding sliding animations.

AI Usage Statement

We confirm that the C++ code, structural design, and technical logic presented in this project were entirely developed by us.

- **Tools Used:** Gemini 3.5 Flash, Gemini 3.1 Pro, Claude Sonnet 4.6

- **Purpose:** AI assistance was utilized strictly for grammar correction, academic phrasing, and structuring the LaTeX formatting of this documentation. Also, used for code formatting for coding section.
- **Accountability:** We fully understand, can explain, and take complete responsibility for every line of code and every technical concept detailed in this report.